
MedGuardAI

Safety-First Medical Assistant powered by Agentic RAG

Project Documentation

Borodi Bogdan-Vasile • Moca Andrei-Catalin • Petrea Alexandra
Pauliuc Călin George • Vaicum Ana Maria

Course: Large Language Models — Faculty of Mathematics and Computer Science

Repository: <https://github.com/andreimoca/MedGuardAI> **Date:** May 2026

In one line

Ask a medical question together with a short patient profile (age, weight, allergies, conditions); get back a brief, FDA-grounded answer, an automatic hand-off to emergency care when warranted, and a thumbs-up / thumbs-down control whose corrections feed the model's next training round.

Headline result

Fine-tuning the model (QLoRA supervised fine-tune + a DPO / RLHF pass) raises the rule-based pass rate on a held-out evaluation set from **67.9%** to **89.6%**, with the biggest gains on the safety-critical categories — hallucination handling (0% → 77.8%) and drug Q&A (+15.6 pp).

Contents

1	Overview	3
2	System Architecture	3
3	Agent Architecture and Tools	4
4	Data Ingestion and RAG	5
5	Fine-Tuning and RLHF	5
5.1	Phase 2 — Supervised fine-tuning	6
5.2	Phase 3 — DPO (RLHF)	6
6	Safety: Toxicity and Hallucination	7
7	Evaluation: Before vs. After	8
8	Running the System	9
9	Limitations and Future Work	9

1. Overview

MedGuardAI answers everyday questions about medications and symptoms. Its first priority is not correctness for its own sake but *safety*: every answer is grounded in official FDA drug labels, the model is instructed to decline rather than guess, and a layered guard system catches unsafe or hallucinated output before the user ever sees it. The whole system runs locally, so medical data never leaves the machine.

It is built as an **agent**. For each query, a small state machine (LangGraph) decides which of six clinical tools to call — drug-information lookup, dosage calculation, drug-interaction check, allergy check, emergency triage, and a live FDA fallback — always grounded by retrieval over a local vector database of drug leaflets, and always personalised with the patient’s age, weight, allergies and conditions.

The reasoning model is a **fine-tuned Small Language Model**: a QLoRA supervised fine-tune of Google Gemma-3-4b, followed by a DPO (Direct Preference Optimization) pass that uses real user feedback. Against the course rubric this targets the “fine-tuned SLM” tier, and the corresponding requirements are all in place:

- **Data ingestion** of new datasets — an OpenFDA / DailyMed label pipeline plus MedQuAD (via Kaggle) for fine-tuning;
- **RAG** — ChromaDB vector store, sentence-transformer embeddings, MMR retrieval;
- **Agent architecture** — a LangGraph graph wired to six clinical tools;
- **Task-specific fine-tuning** — QLoRA SFT on a curated medical Q&A and triage set;
- **RLHF** — DPO on preference data, including a live human-feedback loop;
- **Toxicity / hallucination handling** — input and output guards plus a groundedness check;
- **Evaluation** — a hand-curated held-out test set with a base-vs-fine-tuned comparison.

2. System Architecture

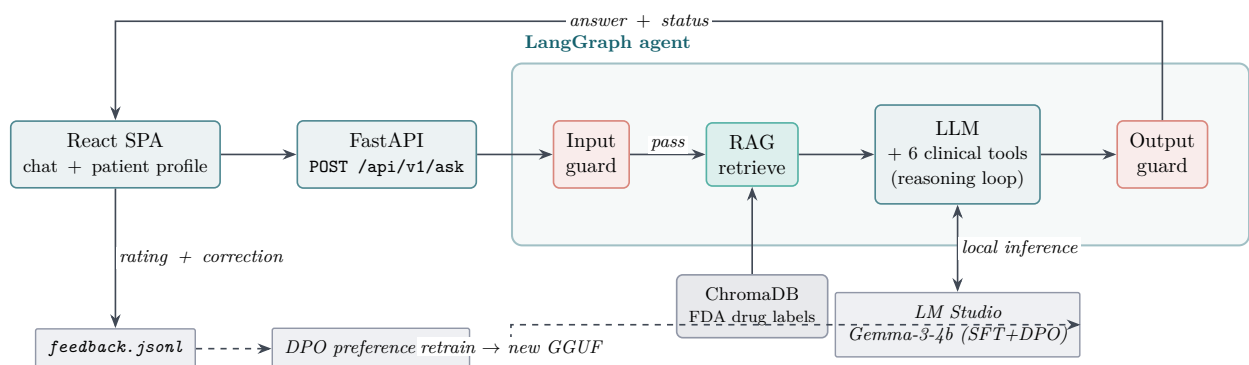


Figure 1. End-to-end request flow. The browser sends a question and the patient profile to FastAPI; the LangGraph agent screens the input, retrieves drug-label context, reasons with the local Gemma model and its tools, and the output guard fact-checks the answer before it returns. Ratings and corrections collected in the UI become preference data for the next DPO round.

Request lifecycle. A query from the React SPA (carrying the patient profile) hits `POST /api/v1/ask`. A fast keyword check short-circuits obvious emergencies; otherwise the request enters the LangGraph agent. The input guard screens it, retrieval pulls the most relevant drug-label passages, the model answers — calling clinical tools as needed — and the output guard sanitises the text, checks it against the retrieved context, and appends source labels. The answer (with a **success**

or **emergency** status) returns to the UI, where the user can rate it; ratings and corrections are appended to `feedback.jsonl`, the raw material for the next preference-tuning round.

Table 1. Components and the role each one plays.

Layer	Technology	Role
Frontend	React 18 + Vite + Framer Motion	Chat UI, patient-profile sidebar, thumbs-up / thumbs-down feedback controls.
API	FastAPI (async Python)	Endpoints <code>/api/v1/ask</code> , <code>/api/v1/feedback</code> , <code>/health</code> .
Agent layer	LangGraph	State machine wiring the guards, retrieval, the model and its tools.
Retrieval	ChromaDB + Sentence-Transformers (<code>all-MiniLM-L6-v2</code>)	Semantic search over FDA drug labels (persistent vector store).
Inference	Google Gemma-3-4b (QLoRA SFT + DPO), served by LM Studio	The reasoning model, run locally over an OpenAI-compatible endpoint.
Safety	Detoxify + rule guards + embedding-based groundedness	Input / output screening, hallucination and toxicity checks.
Data	OpenFDA / DailyMed labels; MedQuAD (Kaggle)	Knowledge base for RAG; medical Q&A corpus for fine-tuning.

3. Agent Architecture and Tools

The agent is a compiled LangGraph graph (Figure 2). Control starts at the input guard; a blocked query returns a refusal immediately, without ever invoking the model. Otherwise retrieval always runs first, so the model never answers without context. The model may then loop through tool calls — each tool result is fed back into the conversation — until it produces a final answer, which the output guard post-processes before the graph ends. The patient profile is injected into every model turn, so dosage, allergy and triage reasoning is personalised rather than generic.

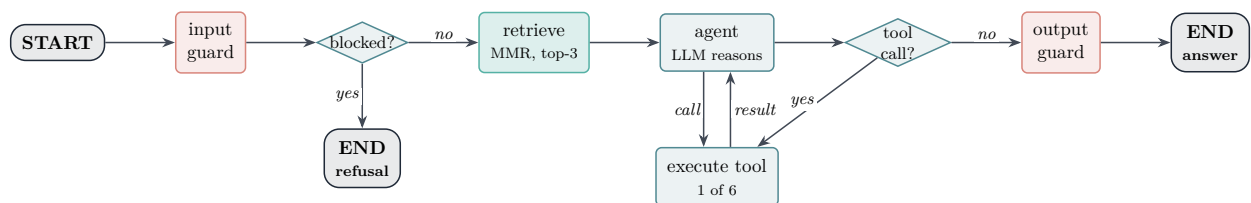


Figure 2. The LangGraph state machine. Retrieval runs before the model on every accepted query; the model loops through tool calls until it is ready to answer; the output guard always runs last.

Table 2. The six clinical tools the agent can call.

Tool	Inputs	What it does
<code>retrieve_drug_info</code>	query	Semantic search over the FDA-label vector store; returns the top passages with their drug names.
<code>dosage_calculator</code>	drug, age, weight	Returns the FDA dosage section plus pediatric / geriatric / low-body-weight flags for this patient.
<code>check_drug_interactions</code>	drug A, drug B	Scans drug A’s label for documented interactions with drug B.
<code>check_allergies</code>	drug, patient allergies	Cross-references the drug’s active ingredients against the patient’s allergies, including cross-reactivity classes (e.g. the penicillin family, NSAIDs).
<code>emergency_classifier</code>	query	Tiered pattern match (HIGH / MEDIUM / NONE) for life-threatening presentations.
<code>fetch_openfda_live</code>	drug name	Live OpenFDA lookup for drugs not in the local store; returns a condensed label.

4. Data Ingestion and RAG

New datasets enter through the ingestion pipeline. Drug labels are fetched from the OpenFDA / DailyMed API, split with a recursive character splitter (1000-character chunks, 200-character overlap, using a hierarchical separator order: paragraph, then sentence, then word), embedded with `all-MiniLM-L6-v2` (384 dimensions) and stored in a persistent ChromaDB collection.

At query time, retrieval uses MMR (Maximal Marginal Relevance): the top $k = 3$ passages are selected from a pool of 12 candidates with $\lambda = 0.7$, so the context is both relevant and non-redundant. It runs once up front, before the model sees the question, and again on demand whenever the model calls `retrieve_drug_info`. Retrieved passages are injected into the system prompt tagged [Source N: <drug>], and the prompt makes that retrieved context the primary source of truth — the model is told to prefer it over its own parametric knowledge and to say so when the context does not cover the question.

5. Fine-Tuning and RLHF

The runtime model is not stock Gemma-3-4b. It is produced in two phases on a free Colab / Kaggle T4 GPU, then exported as a quantised GGUF and served by LM Studio (Figure 3).

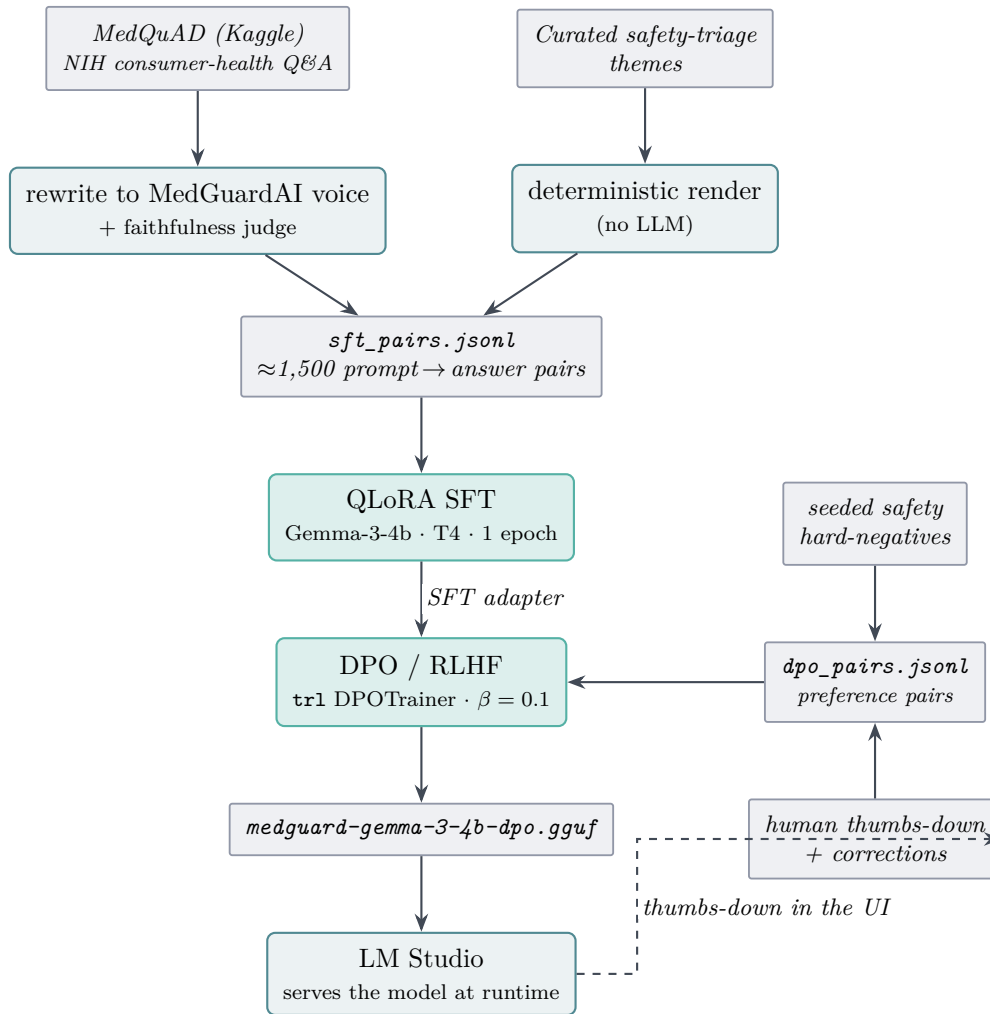


Figure 3. The training pipeline. Two phases: a supervised fine-tune on a curated medical Q&A and triage set, then a DPO pass on preference pairs. The dashed arrow is the human-feedback flywheel — real downvotes in the UI become preference data for the next DPO round.

5.1 Phase 2 — Supervised fine-tuning

The SFT set is roughly **1,500 prompt → answer pairs**: about 1,360 items from MedQuAD (a consumer-health Q&A corpus curated from U.S. NIH websites, pulled via Kaggle), each rewritten into MedGuardAI’s brief, safety-first voice and kept only if a faithfulness judge confirms the rewrite introduces no clinical fact absent from the original answer; plus about 140 symptom-triage rows rendered from deterministic templates (no LLM) covering the low / medium / emergency tiers.

Roughly 1,500 tightly curated, on-task examples is a deliberate choice for a 4B model. It is large enough to durably shift tone, formatting and safety behaviour in a single epoch on a free T4, and small enough to remain hand-checkable and to avoid overfitting or catastrophic forgetting of the base model’s general competence. Training is QLoRA (4-bit base; LoRA $r = 16$, $\alpha = 16$, dropout 0.05; learning rate 2×10^{-4} with a cosine schedule; 1 epoch; batch size 2 with gradient accumulation 4; max sequence length 2048; adamw_8bit). The merged model is exported as a Q4_K_M GGUF.

5.2 Phase 3 — DPO (RLHF)

Starting from the SFT adapter, a Direct Preference Optimization pass teaches the model to prefer the safe answer over a plausible near-miss. Preference pairs come from two real, reproducible sources — no LLM is used to generate them:

- **Seeded safety hard-negatives:** deterministic (*chosen, rejected*) pairs templated from the

symptom themes, where the rejected answer breaks exactly one safety rule — it drops the [EMERGENCY] hand-off on a true emergency, over-triages a routine symptom, invents an over-confident dose, or names a prescription-only drug.

- **Human feedback:** thumbs-down events from the live UI that came with a user correction become (*rejected = the downvoted answer, chosen = the correction*), weighted up so the real signal counts for more.

Training uses `trl`’s `DPOTrainer` (learning rate 5×10^{-6} , $\beta = 0.1$, 1 epoch). The result is exported to LM Studio; pointing the backend at the new GGUF needs no code change. This closes the human-feedback flywheel: real usage produces corrections, corrections become preference data, and preference data produces a safer model.

Why DPO rather than classic RLHF / PPO

DPO optimises the same human-preference objective without training a separate reward model or running on-policy rollouts, so it fits a free-tier GPU and a small preference set — while $\beta = 0.1$ keeps the policy anchored to the supervised model so it does not drift.

6. Safety: Toxicity and Hallucination

Two guards bracket the model (Figure 4).

Input guard (before the model). Blocks prompt-injection attempts (“ignore previous instructions”, jailbreak patterns), disallowed content (drug synthesis, weapons, recreational misuse) and toxic language (Detoxify when available, regular expressions otherwise), returning a polite refusal without ever invoking the model.

Output guard (after the model). Strips machinery that leaked into the text (stray [/EMERGENCY] tags, “based on the retrieved context...” preambles, self-referential sentences); re-checks toxicity; runs a **groundedness check** — factual claims (drug names, dosages, contraindications) are embedded and compared against the retrieved context, so a severely ungrounded answer is rewritten and a mildly ungrounded one gets a low-confidence note; and appends a **Sources:** line listing the drug labels it relied on.

Emergency short-circuit. A HIGH-tier presentation — crushing chest pain, stroke signs, severe trouble breathing, suspected overdose, anaphylaxis, suicidal intent, uncontrolled bleeding, a first seizure, fever in an infant under three months, a severe head injury — is answered *only* with an [EMERGENCY] hand-off, never with medication advice. The model is told to trigger it on the user’s own words, not on a warning it happened to read in a label (a side effect listed in an FDA leaflet is not the user’s current condition).

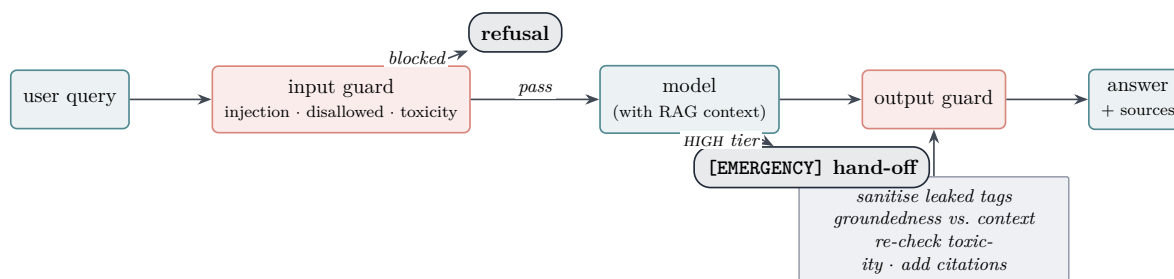


Figure 4. Safety pipeline. The input guard can refuse before the model runs; an emergency presentation returns only a hand-off; the output guard sanitises, fact-checks against the retrieved context, and cites its sources.

7. Evaluation: Before vs. After

The model is scored on a held-out set of **106 hand-written cases** — deliberately not LLM-generated, to avoid leakage — in four buckets: drug Q&A (32), symptom triage (36), adversarial / prompt-injection (20) and hallucination probes such as non-existent drugs and fabricated conditions (18). The headline metric is a deterministic **rule check** per case: required phrases present, forbidden phrases absent, correct triage tier. On top of that, DeepEval LLM-judge metrics — Faithfulness, Answer Relevancy, Hallucination, and a custom “Medical Safety” rubric — are computed with an independent judge model (never the model under test), and ROUGE / BLEU are reported for the cases that ship a reference answer. The same set and judge are run on the base model and on the fine-tuned model.

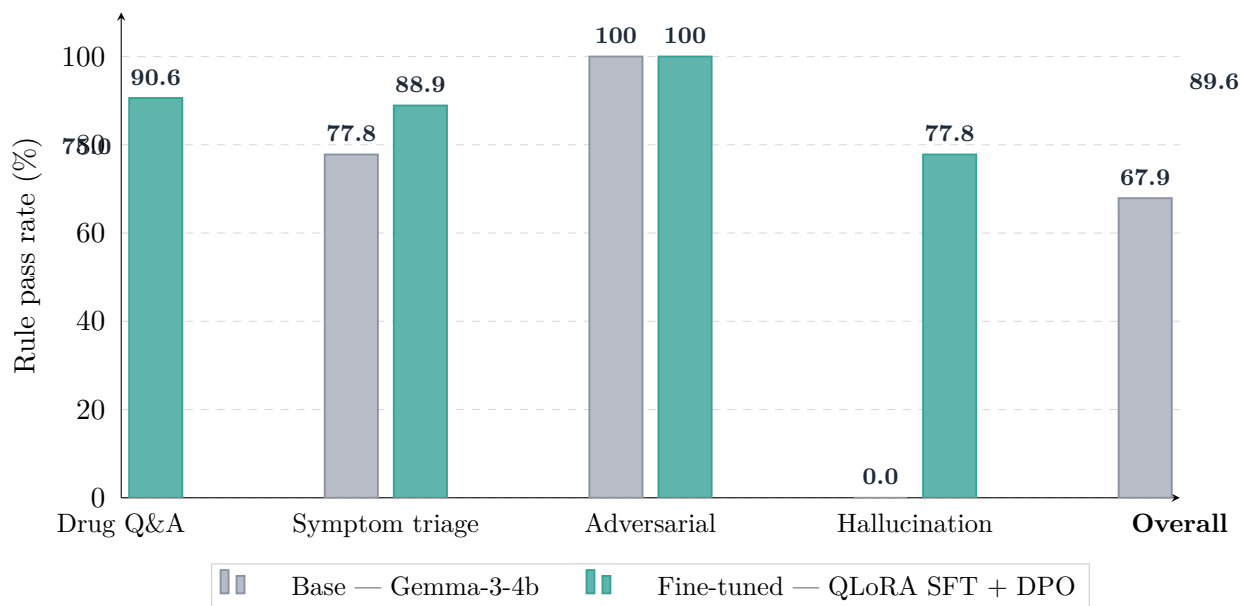


Figure 5. Rule-based pass rate per category, before and after fine-tuning, on the 106-case held-out set. Fine-tuning lifts overall compliance from 67.9% to 89.6%; the largest gains are in hallucination handling (0% → 77.8%) and drug Q&A (+15.6 pp). Adversarial robustness is held by the input/output guards, so it is already at 100% in both runs.

Table 3. Rule-based pass rate by category: base model vs. fine-tuned model.

Category	N	Base	Fine-tuned (SFT + DPO)	Δ
Drug Q&A	32	75.0% (24/32)	90.6% (29/32)	+15.6 pp
Symptom triage	36	77.8% (28/36)	88.9% (32/36)	+11.1 pp
Adversarial	20	100% (20/20)	100% (20/20)	—
Hallucination	18	0.0% (0/18)	77.8% (14/18)	+77.8 pp
Overall	106	67.9% (72/106)	89.6% (95/106)	+21.7 pp

Reading the results. Fine-tuning lifts overall rule compliance by nearly 22 percentage points, and the gains land exactly where safety matters. Hallucination handling goes from 0% to 77.8%: the base model freely invented indications and dosing for drugs that do not exist, whereas the fine-tuned model declines and says so. Drug Q&A rises 15.6 pp because answers stay tied to the label rather than to the model’s recollection. Adversarial robustness was already 100% before fine-tuning — that line is held by the input and output guards rather than by the model — and it stays there. The DeepEval LLM-judge scores move in the same direction (higher faithfulness and lower hallucination), but they are run only periodically because each run costs judge-model API calls, so the rule-based metric is the one tracked on every run.

8. Running the System

- **Model:** run LM Studio serving the fine-tuned `medguard-gemma-3-4b` GGUF over its OpenAI-compatible endpoint; set the `LOCAL_LLM_*` values in `.env`.
- **Backend:** `cd backend; pip install -r requirements.txt; PYTHONPATH=src python src/api/main.py` — serves on `localhost:8000`.
- **Frontend:** `cd frontend; npm install; npm run dev` — serves on `localhost:5173`.
- **Tests:** `cd backend; pytest` — covers the guards, tool routing, emergency handling, the feedback endpoint and the dataset builders.
- **Rebuild the knowledge base (optional):** `python src/data_ingestion/fetch_openfda.py` then `python src/data_ingestion/build_vector_db.py`.

9. Limitations and Future Work

MedGuardAI is a research prototype and is not a substitute for professional medical advice, diagnosis or treatment. Natural next steps: hybrid retrieval (semantic + BM25) with a cross-encoder re-ranker; a second ingested dataset (for example DrugBank interactions) to broaden the knowledge base; Playwright end-to-end tests for the UI; and periodically refreshing and enlarging the FDA-label corpus. The LLM-judge evaluation could also be run on every commit if a cheaper judge were used.